



SDEV2171

Lesson 01: Course Introduction and Mobile App Landscape

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- Explain the scope, structure, and expectations of the course
- Describe the mobile application development landscape and common platform options
- Explain the role of React Native and Expo within modern mobile development
- Identify the types of applications that can be built using React Native

Lesson Roadmap

1. Course structure and expectations
2. Pulse check on prior experience
3. Mobile app development landscape
4. Where React Native and Expo fit
5. Demo of a pre-built Expo app
6. Guided exploration and exit ticket

Course Structure

What students will do in this course

- Set up and use mobile development tools and APIs
- Build and evaluate mobile application code
- Add state, authentication, and web-service features
- Use device hardware in mobile applications

Weekly rhythm

- Concept introduction
- Guided demo or walkthrough
- Hands-on practice
- Follow-up exercise or assessment work

Assessments and Support

Major assessments

- Assessment 1: foundations and core UI
- Assessment 2: state management and feature growth
- Assessment 3: authentication and integration
- Assessment 4: final project

Support model

- In-class walkthroughs
- Practice time and exercises
- Question-driven pacing
- Review of common blockers before they compound

Pulse Check

Quick prompts

- Have you built any app before?
- Have you used JavaScript or React?
- Have you used an API before?
- Have you used an emulator, simulator, or device-testing workflow?

The Mobile App Landscape

Common approaches

- Native: separate apps built with platform-specific tools
- Hybrid: web technologies wrapped in a native shell
- Cross-platform: one codebase targeting multiple platforms

Key question

What tradeoff are we making between control, speed, reuse, and developer experience?

Comparing Approaches

Approach	Strengths	Tradeoffs	Typical Fit
Native	Best platform control, strongest device integration	Two codebases, higher cost	Apps needing deep platform optimization
Hybrid	Fast web-style iteration	UI and performance can feel less native	Content-heavy or simple internal apps
Cross-platform	Shared code and faster delivery	Some platform-specific work still needed	Teams balancing reach and speed

How To Make A Reasonable Guess

Use clues, not certainty

- Does the app seem tightly tied to device hardware?
- Does it behave the same way across platforms?
- Does the UI feel highly customized for one platform?
- Is the app mostly content and forms, or deeply platform-specific?

Where React Native Fits

React Native

- Cross-platform framework using JavaScript and React concepts
- Builds native UI from React component code
- Strong fit for business apps, data-driven apps, and course-scale projects

What students need to understand today

- It is not "just a website in a phone shell"
- It still requires mobile thinking: screens, touch, layout, device constraints

What Expo Adds

Expo helps us move faster

- Project scaffolding and developer tooling
- Device and emulator workflows
- Common APIs and integrations
- Smoother onboarding for a teaching environment

Why this course uses it

- Reduces setup friction
- Lets us focus on mobile concepts earlier
- Gives a stable baseline across student devices

Types of Apps We Can Build

- Trackers and dashboards
- Forms and workflow apps
- Apps using remote APIs
- Authenticated user experiences
- Apps that access location, camera, or other device hardware

Bridge to the rest of the course

Each module expands one of these capabilities.

Demo: Pre-Built Expo App

Watch for these ideas

- Screens: what distinct views exist?
- Components: what UI pieces repeat?
- Navigation: how do you move through the app?
- Feedback preview: what changes on screen after a user action?

Key Terms For Today

- **Screen:** a major view in the app
- **Component:** a reusable UI building block, such as text, a card, or a button
- **Navigation:** how users move from one screen to another
- **Feedback preview:** a visible UI change after input; we will study it more deeply later

Guided Exploration

Student prompts

- List the screens you can identify
- Name at least five visible UI components
- Describe how a user moves from one area to another
- Identify one place where the app gives feedback to the user

Small-group task

Propose one realistic feature improvement and name the affected screens or components.

Key Takeaways

- This course builds from setup to complete mobile features
- Mobile teams choose between native, hybrid, and cross-platform tradeoffs
- React Native gives us a shared-code approach to native mobile apps
- Expo improves workflow and lowers teaching and setup friction

What We Learned Today

- How the course is structured and what students are expected to do
- How native, hybrid, and cross-platform approaches differ
- Why React Native fits a shared-code mobile workflow
- Why Expo is part of the course toolchain
- How to identify screens, components, navigation, and basic UI feedback in an app

Exit Ticket

Students submit

1. One course expectation, support process, or routine you will use this term
2. One type of app React Native could be used to build
3. One thing you now understand about React Native or Expo
4. One question you still have

Next lesson

Lesson 02: tooling setup and environment verification